

AI-POWERED WEB SECURITY PLATFORM: URL, NETWORK AND MALWARE PROTECTION.

A Project Report

Submitted By

Someshwar Nagannath Chopade

2203031260038

in Partial Fulfilment For the Award of

the Degree of

BACHELOR OF TECHNOLOGY

CSE Cyber Security

Under the Guidance of

Prof. Divyeshkumar Hariyani

Professor CSE Department PIET

Parul University Vadodara India



VADODARA

October - 2025



PARUL UNIVERSITY

CERTIFICATE

This is to certify that Project - 2 (303105423) of 7th Semester entitled “AI-Powered Web Security Platform: Url,Network and Malware Protection.” of Group No.21 has been completed by

- Someshwar Nagannath Chopade - 2203031260038

under my guidance in partial fulfillment of the Bachelor of Technology (B.Tech.) in CSE Cyber Security of Parul University in Academic Year 2025- 2026.

Date of Submission :_____

Prof. Divyeshkumar Hariyani,

Project Guide

Prof. Kruti Sutaria,

Project Coordinator

Dr. Daxa Vekariya,

Head of Department,

CSE Cyber Security, PIET,

Parul University.

Acknowledgements

“The single greatest cause of happiness is gratitude.”

-Auliq-Ice

We would like to express our heartfelt gratitude to everyone who has supported, encouraged, and guided us throughout the successful completion of our project.

First and foremost, we extend our deepest appreciation to our parents for being our constant source of strength, motivation, and inspiration. Their unconditional love, support, and faith have been the foundation of our academic journey, and we are truly grateful for their encouragement at every step.

We would also like to express our sincere thanks to the Almighty for blessing us with the strength, wisdom, and perseverance to complete this project successfully. We are truly grateful for the guidance that helped us overcome challenges and achieve this milestone.

Our deepest gratitude goes to our mentor, Prof. Divyeshkumar Hariyani, for his invaluable guidance, encouragement, and continuous support throughout this work. His insightful suggestions and dedicated mentorship have played a vital role in shaping this project, and we sincerely appreciate his patience and commitment.

We would also like to thank Prof. Mukesh Patidar for his valuable assistance and guidance in preparing our research paper. His support has been instrumental in refining our work.

Finally, we express our sincere thanks to Dr. Daxa Vekariya, Head of the Department of CSE (Cyber Security), PIET, for her constant encouragement and support. We are also thankful to all faculty members and peers who directly or indirectly contributed to the successful completion of this project.

We are truly grateful to each one of you for your invaluable support and guidance throughout this journey.

Someshwar Nagannath Chopade
CSE Cyber Security, PIET
Parul University,
Vadodara

Abstract

With the rapid expansion of the internet and the increasing sophistication of cyber threats, online safety has become one of the most critical challenges in today's digital world. Attacks such as phishing, malware distribution, and network intrusions continue to exploit users' trust and system vulnerabilities. Traditional defenses like antivirus software and firewalls often fail to detect new or evolving threats. Therefore, the need for an intelligent, adaptive, and integrated security mechanism has become more urgent than ever.

This project introduces an AI-Powered Web Security Platform, an integrated cybersecurity system designed to detect and prevent threats across three major fronts — URL and Domain Analysis, Network Traffic Anomaly Detection, and Malware File Analysis. Each of these modules functions as a separate yet connected component, working collectively to safeguard users against malicious websites, unsafe network behavior, and harmful files. The system leverages artificial intelligence, rule-based logic, and API intelligence to ensure robust and accurate protection.

The URL and Domain Analysis Module is designed to identify phishing and malicious websites. It extracts multiple features such as domain age, HTTPS presence, IP-based URLs, and token length, which are analyzed using a Random Forest-based machine learning model. To enhance reliability and accuracy, the system integrates Google Safe Browsing API and VirusTotal API. This hybrid approach ensures real-time, high-confidence detection by combining AI-driven classification with trusted global threat intelligence databases.

The Network Traffic Anomaly Detection Module acts as the platform's second line of defense. It utilizes the PyShark library to capture live network packets and monitor traffic patterns in real time. By analyzing attributes such as packet size, duration, protocol type, and frequency, it identifies irregular behaviors that may indicate malicious activity like denial-of-service attacks, data exfiltration, or unauthorized access. When abnormal patterns are detected, the system alerts the user, thereby acting as an early warning mechanism for network-based threats.

The Malware File Analysis Module focuses on ensuring file safety before execution or storage. It evaluates static file features such as entropy, file size, and type, which serve as indicators of malicious intent. A trained machine learning classifier predicts whether a file is malicious or benign based on these extracted attributes. This proactive scanning process helps prevent the download or execution of potentially harmful executables, scripts, or compressed files, providing a critical layer of preemptive defense against malware infections.

The platform is powered by a Flask backend, which manages interactions between all modules and presents results to users via both a web dashboard and a browser extension. The web application allows users to manually input URLs, upload files for scanning, and monitor network activity.

Meanwhile, the browser extension continuously scans visited URLs in real time, displaying results through a simple visual interface. Color-coded feedback—green for safe, yellow for suspicious, and red for malicious—ensures that even non-technical users can understand and act upon the results instantly.

By combining machine learning intelligence with API-based verification, the system offers improved detection accuracy while minimizing false positives. The modular structure allows for scalability, enabling future enhancements such as deep learning–based classifiers, dynamic file sandboxing, and automated intrusion prevention systems. This integration of multiple cybersecurity layers within one framework represents a practical and innovative approach to user protection.

In conclusion, the AI-Powered Web Security Platform delivers a unified, intelligent solution for addressing modern cybersecurity challenges. Through the fusion of machine learning, live traffic analysis, and third-party threat intelligence, it offers proactive, real-time defense across multiple attack surfaces. The system demonstrates how artificial intelligence can transform cybersecurity by shifting focus from passive detection to active prevention, ultimately fostering a safer digital ecosystem for all users.

Table of Contents

Acknowledgements	iii
Abstract	iv
List of Tables	ix
List of Figures	x
1 Introduction	1
1.1 Problem Statement	2
1.2 Scope of the Project	2
1.3 Proposed Solution	2
1.4 Objectives	3
2 Literature Review	4
2.1 Introduction	4
2.2 AI and Machine Learning in Cybersecurity	4
2.2.1 URL and Domain Analysis	5
2.2.2 Network Anomaly Detection	5
2.2.3 Malware File Analysis	5
2.3 Challenges and Research Gaps	6
2.4 Conclusion	6

3	Software Requirements Specification (SRS)	7
3.1	Introduction	7
3.2	Purpose	7
3.3	Scope	8
3.4	Intended Audience	8
3.5	Functional Requirements	8
3.5.1	User Authentication	8
3.5.2	URL & Domain Analysis	8
3.5.3	Network Traffic Anomaly Detection	9
3.5.4	Malware File Analysis	9
3.5.5	Browser Extension Integration	9
3.6	Non-Functional Requirements	9
3.6.1	Performance	9
3.6.2	Security	9
3.6.3	Scalability	10
3.6.4	Compatibility	10
3.7	Software and Hardware Requirements	10
3.7.1	Software Requirements	10
3.7.2	Hardware Requirements	10
3.8	External Dependencies	11
3.9	Conclusion	11
4	System Design	12
4.1	Introduction	12
4.2	System Architecture Overview	12
4.3	Module-Level Design	13
4.3.1	URL and Domain Analysis Module	13

4.3.2	Network Traffic Anomaly Detection Module	13
4.3.3	Malware File Analysis Module	13
4.4	Database Design	16
4.5	Software Stack	17
4.6	User Interface Design	17
4.7	Conclusion	18
5	Methodology	19
5.1	Introduction	19
5.2	User Interaction	19
5.3	Data Processing	20
5.4	AI/ML Analysis	20
5.5	Result Generation	20
5.6	User Action and Prevention	21
5.7	Conclusion	21
6	Implementation	22
6.1	Overview	22
6.2	Testing and Validation	22
6.3	Project Workflow	22
6.4	Experimental Methods	23
7	Conclusion	24
7.1	Practical	24
7.2	Results	25
8	Future Work	28
8.1	Practical	28

List of Tables

3.1	Software Requirements	10
3.2	Hardware Requirements	10
4.1	Software Stack Used in the AI-Powered Web Security Platform	17

List of Figures

4.1	System Architecture of AI-Powered Web Security Platform	13
4.2	Design Flow of URL and Domain Analysis Module	14
4.3	Design Flow of Network Traffic Anomaly Detection Module	15
4.4	Design Flow of Malware File Analysis Module	16
4.5	ER Diagram for the AI-Powered Web Security Platform	17
5.1	Block Diagram	19
7.1	Dashboard Screenshot	25
7.2	File Predicted as Benign Screenshot	25
7.3	File Predicted as Malware Screenshot	26
7.4	url Predicted as Benign Screenshot	26
7.5	url Predicted as Malware Screenshot	26
7.6	Network Analyzer Screenshot	27
7.7	Network Analyzer result Screenshot	27

Chapter 1

Introduction

The digital transformation era has revolutionized the way individuals and organizations interact, communicate, and conduct business online. However, with the rapid growth of internet connectivity, the frequency and sophistication of cyber threats have increased dramatically. Phishing websites, malware-infected files, and network intrusions are among the most common threats compromising user security and privacy. Traditional cybersecurity solutions, such as antivirus software and firewalls, rely primarily on signature-based detection, which often fails to identify emerging or zero-day attacks. To overcome these limitations, there is a growing need for an intelligent, adaptive, and multi-layered approach to cybersecurity.[1]

This project introduces an **AI-Powered Web Security Platform**, an integrated solution designed to provide comprehensive protection against modern cyber threats. The platform operates across three critical layers: URL and Domain Analysis, Network Traffic Anomaly Detection, and Malware File Analysis. Together, these modules deliver a proactive defense mechanism that analyzes URLs in real-time, monitors network activity for anomalies, and evaluates files before they are executed or stored on a system. By integrating Artificial Intelligence (AI) and API-based intelligence, the platform enhances accuracy, transparency, and automation in cybersecurity management.

The platform consists of two main components — a **web application** and a **browser extension**. The web application allows users to manually check URLs, upload files for malware scanning, and monitor real-time network traffic, while the browser extension provides continuous background scanning of web activity. By leveraging AI-driven analysis combined with APIs such as Google Safe Browsing and VirusTotal, this project ensures high detection accuracy and user-friendly interaction. This system not only identifies known malicious entities but also helps in detecting suspicious behavior patterns that may indicate potential cyberattacks.

1.1 Problem Statement

As cyber threats evolve, individuals and organizations face increasing risks from phishing websites, malware downloads, and network-based attacks. Many users unknowingly visit unsafe websites or download infected files, resulting in data breaches, identity theft, or system compromise. Traditional security tools depend on predefined threat signatures and static rules, which cannot detect newly developed or polymorphic attacks. This leads to gaps in detection and leaves users vulnerable to novel cyber threats. The challenge lies in developing a unified and adaptive system capable of identifying, analyzing, and mitigating such threats in real time using intelligent, data-driven approaches.

1.2 Scope of the Project

The scope of this project is to design and implement a real-time AI-driven web security framework that ensures end-to-end protection for internet users. It encompasses three primary modules — (1) URL and Domain Analysis for detecting malicious websites, (2) Network Traffic Anomaly Detection for identifying suspicious activity patterns, and (3) Malware File Analysis for evaluating the safety of uploaded files.[2] The system integrates these modules into a cohesive platform accessible through both a browser extension and a responsive web dashboard. The platform is scalable, modular, and capable of integrating additional security layers in the future, such as behavioral-based intrusion prevention or deep learning–powered malware analysis.

1.3 Proposed Solution

To address the limitations of conventional security tools, this project proposes a hybrid AI-based security model. The solution employs machine learning and trusted APIs to ensure a balance between automation and reliability.

- **URL and Domain Analysis:** The system uses the VirusTotal and Google Safe Browsing APIs to evaluate URLs for malicious content. A browser extension is integrated to alert users before visiting unsafe sites.
- **Network Traffic Anomaly Detection:** Using PyShark, the system captures live network packets and analyzes them to identify anomalies or deviations from normal behavior patterns that may indicate intrusion attempts.

- **Malware File Analysis:** Uploaded files are examined for indicators of compromise based on static features such as entropy, file type, and size. The system classifies files as benign or malicious, helping users avoid potentially harmful downloads.[4]

This approach ensures proactive defense through intelligent automation, thereby reducing dependence on human intervention and manual monitoring.

1.4 Objectives

The main objectives of this project are:

1. To develop an AI-based classifier that accurately distinguishes between safe and malicious URLs.
2. To implement a real-time network traffic analysis system capable of identifying abnormal or unsafe network behaviors.
3. To build a malware file analysis module that evaluates files before execution, preventing potential infections.
4. To integrate the above modules into a single, user-friendly web application with a supporting browser extension.
5. To enhance cybersecurity awareness by providing users with transparent and easily interpretable results.

Chapter 2

Literature Review

2.1 Introduction

With the exponential growth of digital connectivity and online services, cyber threats such as phishing, malware, ransomware, and network intrusions have become increasingly complex and adaptive. Traditional signature-based security mechanisms, though once effective, are now insufficient to detect zero-day exploits or polymorphic attacks that constantly evolve their behavior. In response to these limitations, Artificial Intelligence (AI) and Machine Learning (ML) have emerged as revolutionary tools for strengthening cybersecurity frameworks. By enabling pattern recognition, anomaly detection, and predictive analytics, AI-based systems can identify and mitigate emerging threats in real time.

This chapter provides a comprehensive review of existing literature on AI-powered cybersecurity solutions, with particular emphasis on three critical areas — **URL and Domain Analysis**, **Network Traffic Anomaly Detection**, and **Malware File Analysis**. The goal is to understand how these technologies enhance the accuracy, adaptability, and scalability of modern cybersecurity solutions.

2.2 AI and Machine Learning in Cybersecurity

Machine Learning algorithms have demonstrated exceptional capabilities in identifying malicious patterns that may escape traditional systems. By learning from historical data and continuously adapting to new inputs, AI models can recognize deviations in normal behavior indicative of cyberattacks. Several research efforts have explored the role of ML in web security, malware detection, and network protection.

2.2.1 URL and Domain Analysis

Research by **Aljabri et al. (2022)** provided a comprehensive evaluation of machine learning methods for detecting malicious URLs, emphasizing the inadequacy of conventional blacklist-based detection. The study compared multiple algorithms including *Convolutional Neural Networks (CNNs)*, *XGBoost*, *Long Short-Term Memory (LSTM)*, and *Support Vector Machines (SVMs)*, achieving accuracy rates as high as 99.98%. Their findings demonstrate that deep learning models can capture subtle textual and lexical patterns within URLs that are indicative of phishing or malicious intent.[5] However, they also noted the lack of research on region-specific malicious domains, such as Arabic or vernacular websites, which presents a potential research gap for more inclusive and adaptive URL threat detection systems.

2.2.2 Network Anomaly Detection

In a study by **Kahraman Kostas (2018)**, various machine learning algorithms were applied to the **CICIDS2017** dataset to detect anomalous patterns in network traffic. The research evaluated classifiers such as *K-Nearest Neighbors (KNN)*, *Decision Tree (ID3)*, *Random Forest*, *AdaBoost*, *Naive Bayes*, *Quadratic Discriminant Analysis (QDA)*, and *Multi-Layer Perceptron (MLP)*. The accuracy achieved ranged between 83% and 97%, with ensemble methods like Random Forest and boosting techniques showing the most consistent results. This work underscored the significance of ML-based intrusion detection systems (IDS) in identifying both encrypted and unencrypted malicious traffic efficiently. It also highlighted the need for real-time adaptability and low false-positive rates in operational environments.[6]

2.2.3 Malware File Analysis

The work of **Rami Sihwail, Khairuddin Omar, and K.A.Z. Ariffin (2018)** offered a detailed survey of malware analysis methodologies, including static, dynamic, hybrid, and memory-based techniques. Static analysis involves examining file attributes—such as entropy, structure, and metadata—without executing the file, enabling rapid but limited detection. Dynamic analysis, in contrast, runs the file within a sandbox environment to monitor its runtime behavior, making it more accurate but computationally expensive. Hybrid approaches combine these two techniques for enhanced reliability, while memory analysis detects sophisticated in-memory threats that evade conventional methods. The study emphasizes the importance of integrating hybrid models and behavioral analysis for robust malware detection. [7]

These insights support the approach taken in this project's **Malware File Analysis Module**,

which evaluates file entropy, size, and type to classify malicious intent before execution.

2.3 Challenges and Research Gaps

Despite the progress in AI-based cybersecurity, several critical challenges persist that hinder real-world deployment and scalability:

1. **False Positives and Negatives:** ML models can sometimes misclassify benign behavior as malicious or fail to detect subtle threats, leading to unreliable protection outcomes.
2. **Adversarial Attacks:** Attackers may manipulate model inputs to deceive AI systems — a growing threat known as adversarial machine learning.
3. **Data Scarcity:** Obtaining large, high-quality, labeled datasets for training cybersecurity models remains difficult, particularly for zero-day attacks.
4. **Real-Time Efficiency:** Some AI models are computationally heavy, making real-time deployment challenging without performance trade-offs.
5. **Model Interpretability:** Many deep learning models function as “black boxes,” providing limited transparency in decision-making, which reduces trust and usability in enterprise environments.

Addressing these gaps requires developing hybrid AI architectures, improved data curation methods, and real-time adaptive mechanisms.

2.4 Conclusion

The integration of **URL analysis**, **network anomaly detection**, and **malware file analysis** forms a **multi-layered security architecture** that ensures comprehensive threat detection and prevention. These methods complement one another—while URL analysis blocks phishing attempts at the entry point, anomaly detection safeguards ongoing network communication, and malware analysis prevents post-download infections.

Although these technologies have achieved impressive success, continued research is needed to improve **accuracy**, **speed**, and **resilience** against adversarial manipulation. Future innovations in explainable AI, federated learning, and automated retraining pipelines could make AI-powered cybersecurity platforms more transparent, ethical, and globally deployable.

Chapter 3

Software Requirements Specification (SRS)

3.1 Introduction

The Software Requirements Specification (SRS) defines the complete functional and non-functional requirements of the **AI-Powered Web Security Platform**. It describes the overall architecture, system functionalities, user expectations, and technical requirements necessary for system design, implementation, and deployment. The goal of this project is to provide a unified platform that safeguards users from online threats such as phishing, malware, and network intrusions. The system integrates Artificial Intelligence (AI), rule-based logic, and API intelligence to provide accurate and real-time cyber threat detection.

3.2 Purpose

The **AI-Powered Web Security Platform** is developed to detect, analyze, and prevent online threats using multiple integrated security layers. It is designed to protect users from malicious websites, unsafe downloads, and suspicious network activity. The system combines Machine Learning (ML) and API-based detection for accuracy, providing transparency and security to end users. It consists of three primary modules:

- **URL & Domain Analysis:** Identifies phishing, unsafe, and deceptive websites using both ML-based analysis and Google Safe Browsing API.[8]
- **Network Traffic Anomaly Detection:** Monitors real-time network packets using PyShark to detect abnormal traffic patterns or intrusions.[9]
- **Malware File Analysis:** Scans uploaded files for potential malware by examining static attributes such as entropy, size, and file type using ML models.[10]

3.3 Scope

The project includes both a web application and a browser extension, providing multi-layered online security:

- A **web application** for users to manually check URLs, scan files, and view network anomaly reports.
- A **browser extension** that automatically checks every URL visited by the user in real time and alerts them if a website is unsafe or suspicious.
- A **Flask-based backend** integrating ML models and APIs for intelligent security analysis.
- Integration with the **Google Safe Browsing API** for web threat verification and **PyShark** for live network traffic analysis.

The system aims to provide a transparent, automated, and user-friendly cybersecurity solution accessible to individuals, developers, and organizations.

3.4 Intended Audience

This platform is designed for a wide range of users and stakeholders:

- Cybersecurity professionals seeking automated threat monitoring tools.
- Researchers and students studying applied AI in cybersecurity.
- General internet users looking for real-time browsing protection.
- Organizations and network administrators monitoring network anomalies.

3.5 Functional Requirements

3.5.1 User Authentication

- Users can securely log in or register (optional for public scanning).
- Admins can manage ML models, logs, and system configurations.

3.5.2 URL & Domain Analysis

- Accepts a URL input for analysis.
- Uses both ML-based models and Google Safe Browsing API.

- Displays results as **Safe**, **Suspicious**, or **Malicious**.
- Generates visual outputs such as pie charts showing detection statistics.

3.5.3 Network Traffic Anomaly Detection

- Captures live network packets using PyShark.
- Analyzes packets based on protocol, size, and frequency.
- Detects anomalies or suspicious traffic behavior.
- Displays structured results in the dashboard.

3.5.4 Malware File Analysis

- Allows users to upload files for scanning before execution.
- Extracts file features such as entropy, size, and type.
- Uses ML models to classify files as **Malicious** or **Safe**.
- Displays threat details and confidence metrics.

3.5.5 Browser Extension Integration

- Monitors URLs visited in the browser.
- Sends URL data to the backend for AI-based analysis.
- Displays color-coded alerts (green = safe, red = malicious, yellow = suspicious).

3.6 Non-Functional Requirements

3.6.1 Performance

- URL analysis response time under 2 seconds.
- System supports over 100 concurrent scan requests.

3.6.2 Security

- Implements AES-256 encryption for secure data handling.
- Uses secure API authentication for Google Safe Browsing requests.
- Restricts file uploads to prevent code injection.

3.6.3 Scalability

- Supports concurrent users via multi-threaded Flask server.
- Can be deployed using Docker and scaled across cloud environments.

3.6.4 Compatibility

- Compatible with Windows, macOS, and Linux.
- Browser extension supports Chrome, Firefox, and Edge.
- Web app optimized for desktop and mobile devices.

3.7 Software and Hardware Requirements

3.7.1 Software Requirements

Component	Technology Used
Frontend	HTML, CSS, JavaScript
Backend	Flask (Python)
Database	SQLite / PostgreSQL
ML Models	Random forest, Isolation Forest
Network Monitoring	PyShark / Wireshark
Browser Extension	JavaScript, Chrome Extension API
API Services	Google Safe Browsing API, VirusTotal API,

Table 3.1: Software Requirements

3.7.2 Hardware Requirements

Component	Minimum Specification
Processor	Intel Core i5 / AMD Ryzen 5
RAM	8 GB (Recommended: 16 GB)
Storage	50 GB free space
Internet	Stable connection for API calls

Table 3.2: Hardware Requirements

3.8 External Dependencies

The system relies on several external services and tools for enhanced performance and real-time threat detection:

- **Google Safe Browsing API:** For URL reputation and phishing detection.
- **VirusTotal API:** For visualization and comparative analysis.
- **PyShark / TShark:** For capturing and analyzing live network packets.

3.9 Conclusion

This SRS defines all key aspects of the AI-Powered Web Security Platform. The system integrates ML, network monitoring, and API-based intelligence to ensure multi-layered protection. The requirements outlined will serve as a foundation for implementation, testing, and future scalability.

Chapter 4

System Design

4.1 Introduction

The System Design chapter provides a detailed overview of how the AI-Powered Web Security Platform is structured and how its components interact to ensure secure, efficient, and reliable performance. This design phase bridges the gap between system requirements and implementation, transforming the conceptual model into a structured, modular architecture. The system is developed with a focus on modularity, scalability, and security, ensuring that all three modules — URL & Domain Analysis, Network Traffic Anomaly Detection, and Malware File Analysis — operate cohesively through a unified backend.

4.2 System Architecture Overview

The architecture of the platform follows a client-server model, integrating both a web-based dashboard and a browser extension with a Flask backend. The system employs a multi-layered security mechanism where inputs from users or background scans are processed by AI-based models and APIs for accurate threat assessment.

- The **Frontend Layer** includes a web dashboard for manual analysis and a browser extension for real-time URL scanning.
- The **Backend Layer** (Flask) manages data flow between modules, APIs, and databases while coordinating security analysis.
- The **AI/ML Layer** performs classification and anomaly detection using trained models.
- The **API Integration Layer** connects with Google Safe Browsing and VirusTotal for external threat verification.

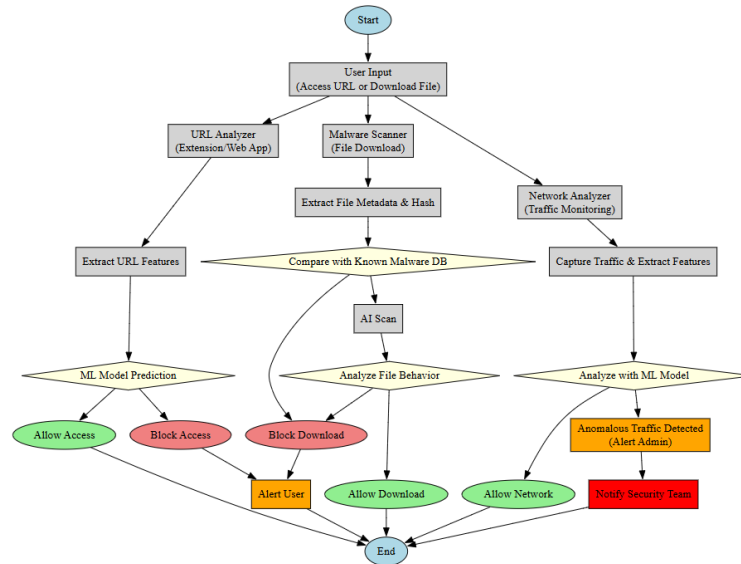


Figure 4.1: System Architecture of AI-Powered Web Security Platform

4.3 Module-Level Design

Each of the three security modules performs a distinct but interconnected function, ensuring comprehensive protection across multiple attack vectors.

4.3.1 URL and Domain Analysis Module

This module evaluates URLs to detect phishing or malicious websites. The user enters a URL, which is processed using a combination of ML-based feature analysis and external API checks. The system extracts URL features such as length, presence of special symbols, domain age, and protocol type. It then applies the trained Random Forest model to classify the URL as safe, suspicious, or malicious. To enhance accuracy, Google Safe Browsing and VirusTotal APIs are used for secondary verification.[11]

4.3.2 Network Traffic Anomaly Detection Module

This module analyzes live network traffic to detect anomalies and intrusions. PyShark captures and inspects packets to identify deviations from normal patterns, such as unusual IP behavior, port scanning, or data surges. Instead of ML-based predictions, this module relies on rule-based detection and statistical thresholds to classify traffic as safe or suspicious. The analyzed results are displayed to the user with alerts highlighting abnormal activities.[12]

4.3.3 Malware File Analysis Module

The Malware File Analysis module provides proactive protection by scanning files before they are downloaded. It extracts static attributes such as entropy, file size, and type to predict whether the

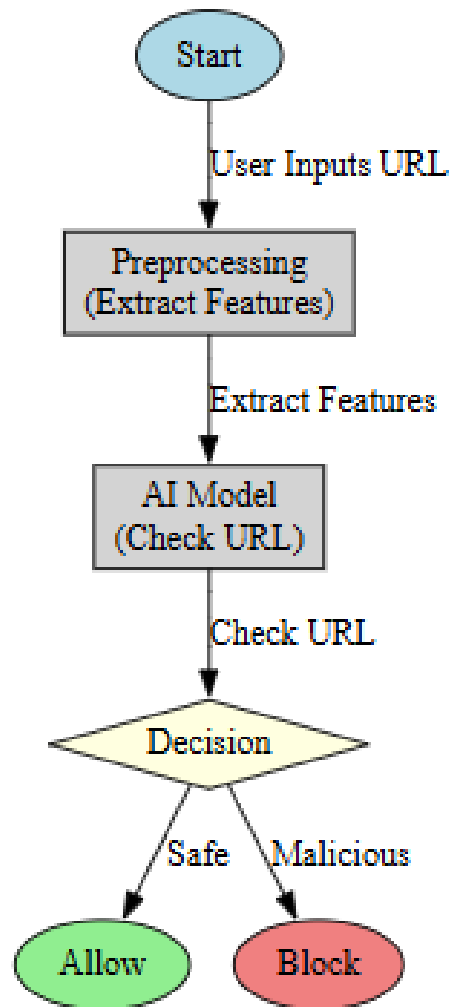


Figure 4.2: Design Flow of URL and Domain Analysis Module

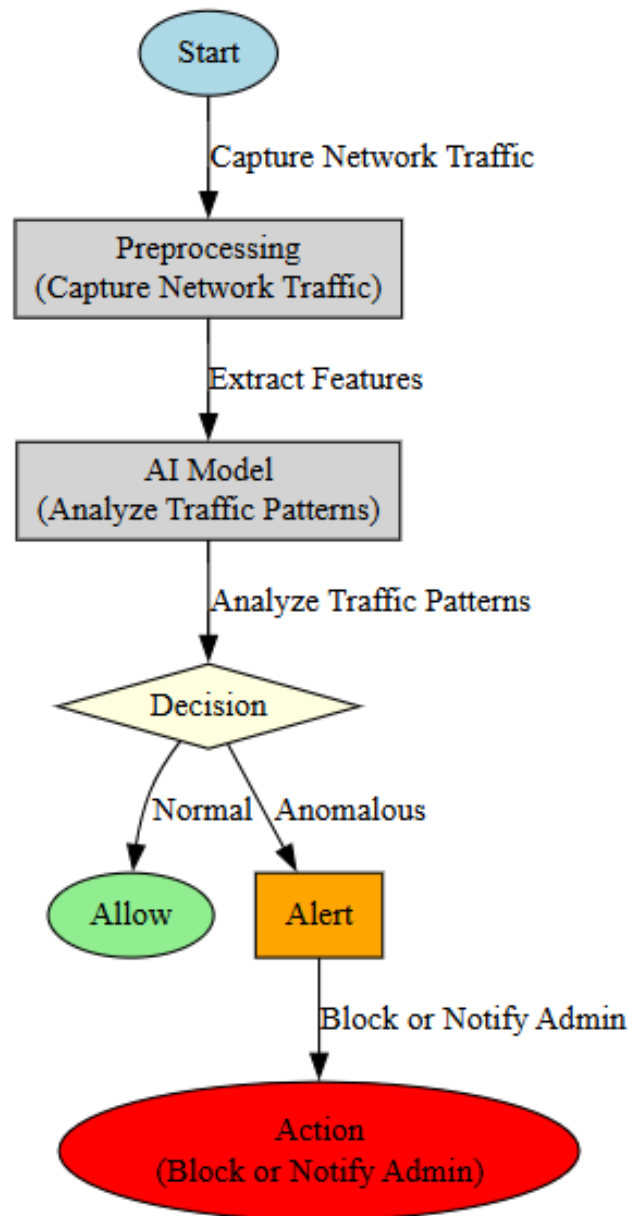


Figure 4.3: Design Flow of Network Traffic Anomaly Detection Module

file is malicious or benign. This hybrid analysis (static + heuristic) helps users avoid downloading infected files, offering an early defense before execution. Files flagged as malicious are automatically blocked or quarantined.[13]

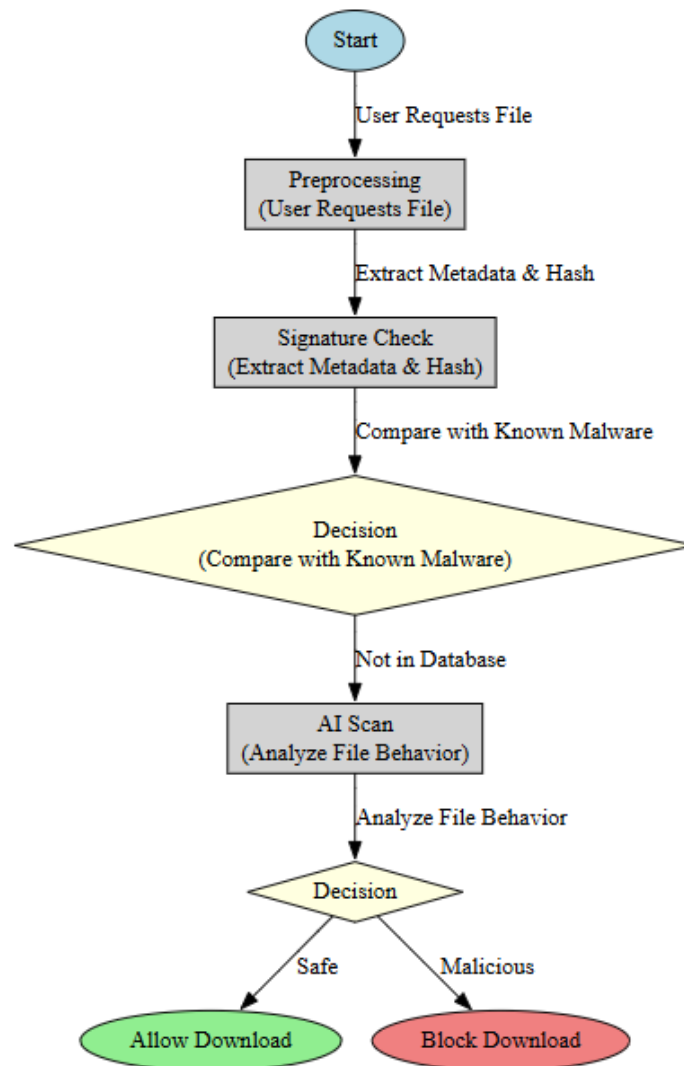


Figure 4.4: Design Flow of Malware File Analysis Module

4.4 Database Design

The system utilizes a relational database (SQLite or PostgreSQL) to store URLs, scan logs, user activity, and threat reports. Each table maintains structured data to enable traceability and analytics.

- **URL_Scans:** Stores URLs, classification results, and timestamps.
- **Network_Logs:** Records captured packets and flagged anomalies.
- **File_Analysis:** Maintains file hashes, entropy values, and scan verdicts.

- **User Activity:** Logs interactions performed via the web dashboard or extension.

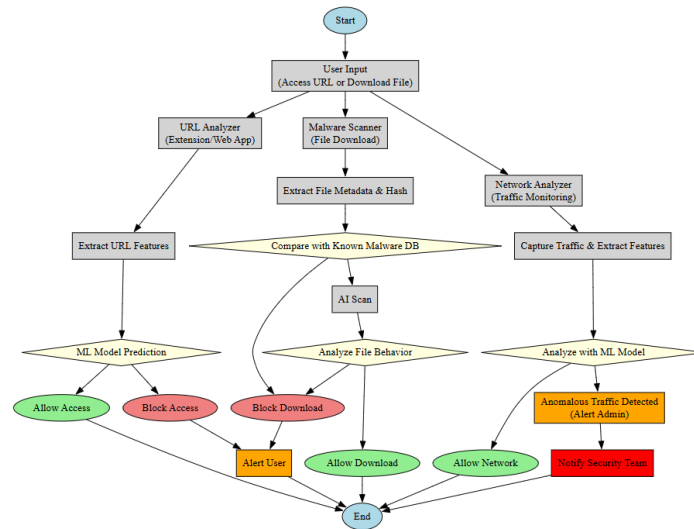


Figure 4.5: ER Diagram for the AI-Powered Web Security Platform

4.5 Software Stack

The following table presents the technologies and frameworks used in the project.

Component	Technologies Used
Frontend	HTML, CSS, JavaScript
Backend	Flask (Python)
Database	PostgreSQL / SQLite
Machine Learning	Random forest, Isolation Forest
Network Monitoring	PyShark, Wireshark
APIs Integrated	VirusTotal API, Google Safe Browsing API
Browser Extension	JavaScript, Chrome Extension API

Table 4.1: Software Stack Used in the AI-Powered Web Security Platform

4.6 User Interface Design

The user interface (UI) is designed for simplicity, clarity, and responsiveness. The web dashboard allows users to:

- Enter URLs or upload files for scanning.

- View categorized results with visual charts.
- Monitor network anomaly results.

The browser extension runs continuously in the background, automatically scanning websites in real-time and alerting users before visiting unsafe pages.

4.7 Conclusion

This chapter presented the complete system design of the AI-Powered Web Security Platform, detailing its modular architecture, database structure, and technology stack. The design ensures modular integration of all three security components—URL analysis, network anomaly detection, and file malware analysis—through a unified backend. The next chapter focuses on implementation, covering system integration, testing, and experimental validation.

Chapter 5

Methodology

5.1 Introduction

This chapter presents the overall methodology, workflow, and system architecture used in developing the AI-Powered Web Security Platform. The platform integrates three core modules—URL and Domain Analysis, Network Traffic Anomaly Detection, and Malware File Analysis—to deliver multi-layered protection. The methodology explains how user interactions, data processing, AI-based analysis, and threat prevention are coordinated within a unified system architecture.

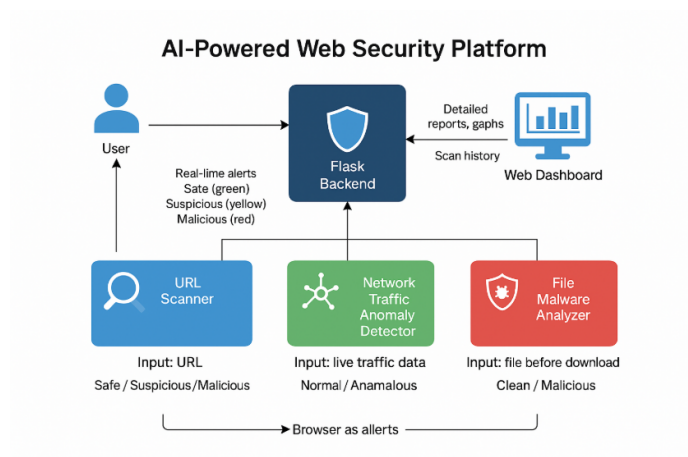


Figure 5.1: Block Diagram

5.2 User Interaction

The platform is designed to provide seamless and real-time protection to users across both web and browser environments.

- Users can enter a URL or upload a file for manual scanning via the web interface.
- The browser extension automatically monitors and scans URLs in real-time while the user

browses.

- Alerts are displayed instantly if any threat is detected, improving user awareness and response time.

5.3 Data Processing

Data processing forms the backbone of the platform's analytical workflow.

- The entered URL is sent to the Flask backend for feature extraction and analysis.
- Network traffic is continuously monitored using PyShark and packet capture tools to identify anomalies.
- Uploaded files are scanned prior to download, ensuring proactive detection of potential malware.

5.4 AI/ML Analysis

The system leverages Artificial Intelligence (AI) and Machine Learning (ML) models to identify malicious behavior. [14]

- **URL Analysis:** The URL is evaluated using Random Forest and supported by Google Safe Browsing or VirusTotal API to ensure higher accuracy.
- **Network Analysis:** Network behavior is monitored using PyShark and analyzed with anomaly detection algorithms to flag suspicious activities.
- **Malware File Analysis:** Files are inspected using static features such as entropy, size, and file type, helping the model predict whether the file is malicious or benign.

5.5 Result Generation

After the AI analysis, the system produces comprehensive results for the user.

- Each module categorizes its output as *Safe*, *Suspicious*, or *Malicious*.
- Results are displayed in real-time through visual indicators and detailed reports.
- If a potential threat is found, immediate alerts are sent through the dashboard and browser extension.

5.6 User Action and Prevention

The platform assists users in making informed decisions based on detected threats.

- If the resource is **Safe**, users can proceed normally.
- If the result is **Suspicious**, users are advised to exercise caution.
- If the URL or file is **Malicious**, the system blocks access or prevents the download automatically.

5.7 Conclusion

This chapter presented the detailed methodology, of the AI-Powered Web Security Platform. By combining intelligent analysis, real-time monitoring, and user-focused design, the platform ensures a proactive approach to threat detection and mitigation. The following chapter discusses the system design and architecture in detail.

Chapter 6

Implementation

6.1 Overview

Implementation focuses on integrating all modules into a unified and responsive system.

- The browser extension monitors active URLs and warns users in real-time.
- The web application allows users to manually submit URLs or files for security checks.
- The Flask backend processes inputs, communicates with APIs, and generates final risk assessments.

6.2 Testing and Validation

The system is tested to ensure robustness and accuracy using multiple datasets and test environments.

- Phishing URL datasets, network logs, and malware samples are used for evaluation.[15]
- Each module undergoes functional and stress testing to assess performance under load.
- Real-time validation ensures that detection models produce accurate classifications consistently.

6.3 Project Workflow

The overall project workflow is outlined below:

1. **Data Collection:** Datasets containing labeled malicious and benign URLs, network logs, and malware signatures are gathered.[16]
2. **Model Training:** ML models are trained using supervised learning for classification tasks.[17]

3. **Web Application Development:** A responsive interface is built for manual URL and file analysis.[19]
4. **Browser Extension Development:** Real-time background scanning and alerting mechanisms are implemented.
5. **Testing and Evaluation:** The complete system is validated for performance, scalability, and detection accuracy.

6.4 Experimental Methods

The following experimental methods are applied during development and validation:

- **Supervised Learning:** Used for URL classification based on extracted features.
- **Rule-Based Anomaly Detection:** Applied for detecting suspicious patterns in network traffic using PyShark.
- **Static File Analysis:** Evaluates file entropy, size, and type for identifying malware.[20]
- **API-Integrated Threat Verification:** Cross-verifies results using Google Safe Browsing and VirusTotal.

Chapter 7

Conclusion

7.1 Practical

The AI-Powered Web Security Platform developed in this project successfully integrates multiple security modules into a cohesive and responsive system. The practical outcomes of the project include:

- **Real-Time URL Protection:** The browser extension provides instantaneous feedback on the safety of websites, alerting users to malicious or suspicious URLs and significantly reducing the risk of phishing attacks.[21]
- **Comprehensive Web Application:** Users can manually submit URLs or files for analysis, receiving detailed reports that combine machine learning predictions and API-based threat verification. This feature enhances transparency and allows for informed decision-making.
- **Robust Backend Processing:** The Flask backend efficiently handles multiple data streams, integrates with external APIs such as Google Safe Browsing, and leverages trained ML models to provide accurate risk assessments.
- **Network Traffic and Malware Analysis:** The system effectively identifies anomalous network behavior and malware threats, ensuring a multi-layered defense against cyber threats.
- **Scalability and Reliability:** Through rigorous testing and validation on diverse datasets, the platform has demonstrated high reliability and consistent performance, making it suitable for practical deployment in real-world scenarios.

Overall, this project highlights the feasibility of combining machine learning, rule-based detection,[22] and API-integrated verification into a unified security framework. The platform not

only enhances individual user safety but also provides a foundation for future expansion, such as integrating additional threat intelligence feeds, real-time alert dashboards, and AI-powered automation for enterprise-scale deployment.

In conclusion, the project successfully meets its objectives of creating a practical, intelligent, and user-friendly system for proactive web security, showcasing the potential of AI-driven solutions in the cyber security domain.

7.2 Results

The following results demonstrate the functionality and effectiveness of the three core modules of the platform. Screenshots of detections for both safe and malicious inputs can be included here:



Figure 7.1: Dashboard Screenshot

- **Module 1 – URL Analysis:** Screenshots showing safe and phishing URL detection from both the browser extension and web application.

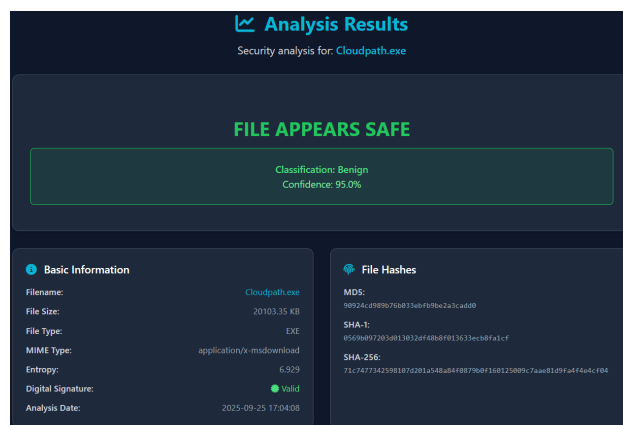


Figure 7.2: File Predicted as Benign Screenshot

- **Module 2 – Network Traffic Anomaly Detection:** Screenshots of network monitoring results, highlighting normal vs suspicious traffic patterns.

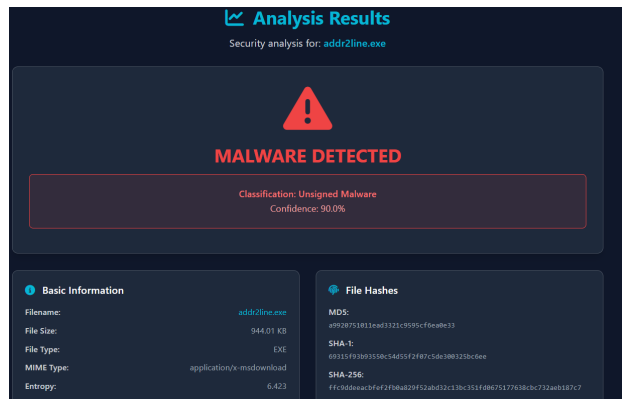


Figure 7.3: File Predicted as Malware Screenshot

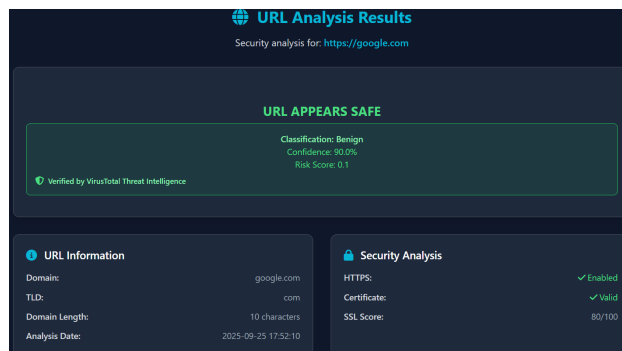


Figure 7.4: url Predicted as Benign Screenshot

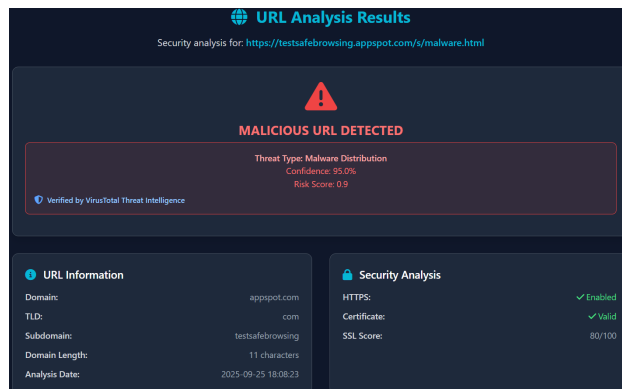


Figure 7.5: url Predicted as Malware Screenshot

- **Module 3 – File Malware Analysis:** Screenshots of file analysis showing safe files versus identified malicious files, including detailed risk reports.



Figure 7.6: Network Analyzer Screenshot

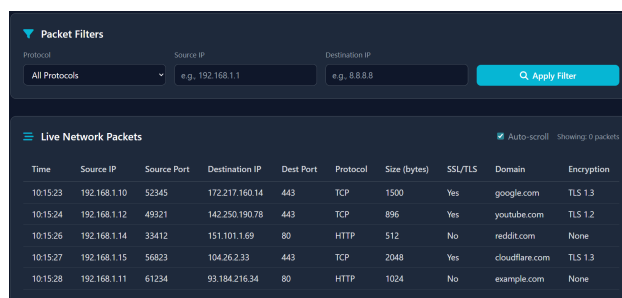


Figure 7.7: Network Analyzer result Screenshot

These results visually validate the system's real-time threat detection capabilities and demonstrate its reliability across multiple scenarios. Screenshots should be annotated with brief descriptions to indicate the type of detection and the module used.

Chapter 8

Future Work

8.1 Practical

While the AI-Powered Web Security Platform provides robust protection against phishing URLs, network anomalies, and malware, there are several opportunities to further enhance its capabilities. Future developments could include:

- **Integration of Additional Threat Intelligence Sources:** Incorporating more APIs and real-time threat feeds, such as threat intelligence platforms and community-driven blacklists, could improve detection accuracy and coverage.
- **Enhanced Machine Learning Models:** Exploring advanced deep learning techniques, such as graph neural networks for URL structure analysis or recurrent neural networks for network traffic patterns, can increase predictive performance and reduce false positives.
- **Real-Time Dashboard and Analytics:** Developing a centralized dashboard for users and administrators to monitor threats, view trends, and generate reports would improve usability and provide actionable insights.
- **Automated Threat Response:** Implementing automated mitigation strategies, such as blocking malicious downloads or quarantining suspicious traffic in real time, would make the system more proactive rather than purely diagnostic.
- **Cross-Platform and Mobile Support:** Extending browser extension compatibility to multiple browsers and developing mobile apps could expand accessibility for a wider range of users.
- **Privacy-Preserving Features:** Integrating privacy-preserving machine learning approaches, such as federated learning, would allow the system to improve detection capabilities without

compromising user data.

- **Enterprise-Level Deployment:** Adapting the platform for large-scale organizational networks, with features like centralized policy management and real-time alerts for IT teams, would make it suitable for corporate cybersecurity applications.

Overall, the proposed future enhancements aim to make the platform more intelligent, scalable, and user-friendly, ensuring comprehensive protection against evolving cyber threats while maintaining performance and reliability.

Bibliography

- [1] Young-Sik Jeong, Yi Pan, J. Kang. (2019). *Malware Classification Using Machine Learning*. Advances in Computer Science and Ubiquitous Computing, Springer
- [2] Wahid Urban, Mohammad Mahdi Darbi, Mohammad Isadi. (2020). *Fishing detection* using machine learning techniques. Arxiv Preprint Arxiv: 2009.111116.
- [3] Iraj Sadegh Amiri, etc. Akanbi, E. Fazeldehkord. (2018). *a machine learning approach for fishing detection and defense*. Sinkgrass, Elcewear. ISBN: 978-0-12-802927-5.
- [4] O.K. Sahingose, Abubekir Buber, O. Dameer, Banu Diri. (2019). *Machine Learning based Fishing Detection* from url. Expert systems with applications, 117, 345–357.
- [5] M. Aijabri, H. S. Altamimi, S. Albelali and M. al-Hhabi. (2022). *detecting malicious urls using machine learning techniques: Review and Search Directions*. IEEE access. DOI: 10.1109/Access.2022.3222307.
- [6] M.S.Akhtar,T.Feng. (2023).*Evaluation of machine learning algorithms for malware detectionsensors*. 10.3390/s23020946.
- [7] S.I. Rimon, M.M. Haque. (2022).*malware detection and classification using hybrid machine learning algorithm*.intelligent computing and optimization, pp. 419-428.10. 1007/978-3-031-19958-5-39
- [8] A. Hussain, M. Asif, M. Ahmad, T. Mahmood. (2022).*malware detection using machine learning algorithms for windows platform*.(ICITA 2021), pp. 619-632. 10.1007/978-981-16-7618-5_53.
- [9] J. Gutierrez, E. Villarroel. (2024).*dynamic malware analysis using machine learning based detection algorithms*.Interfases,19. DOI: 10.26439/interfases2024.n19.7097.

- [10] S. Rkhouya, K. Chougali. (2021). *malware detection using a machine learning based approach*. (IJITAS), 3(4), pp. 167-171. DOI: 10.52502/ijitas.v3i4.172.
- [11] Weibao Zhang, Joan P. Lazaro. (Recent). *A Survey on Network Security Traffic Analysis and Anomaly Detection Techniques*. IJETAA.
- [12] Yalu Wang, Zhijie Han, Jie Li, Xin He. (2023). *BS-GAT: Behavior Similarity Based Graph Attention Network for Network Intrusion Detection*. arXiv preprint.
- [13] Tajuddin Manhar Mohammed, Lakshmanan Nataraj, Satish Chikkagoudar, Shivkumar Chandrasekaran, B. S. Manjunath. (2021). *Malware Detection Using Frequency Domain-Based Image Visualization and Deep Learning*. arXiv preprint arXiv:2101.10578.
- [14] Ahmed Bensaoud, Jugal Kalita, Mahmoud Bensaoud. (2024). *A Survey of Malware Detection Using Deep Learning*. Machine Learning with Applications, **16**.
- [15] Julian Busch, Anton Kocheturov, Volker Tresp, Thomas Seidl. (2021). *NF-GNN: Network Flow Graph Neural Networks for Malware Detection and Classification*. arXiv preprint arXiv:2103.03939.
- [16] Manzil H.H.R., Manohar Naik. (2023). *Android Malware Category Detection Using a Novel Feature Vector-Based Machine Learning Model*. Cybersecurity. SpringerOpen.
- [17] Sachith Seneviratne, Ridwan Shariffdeen, Sanka Rasnayaka, Nuran Kasthuriarachchi. (2022). *Self-Supervised Vision Transformers for Malware Detection*. arXiv preprint arXiv:2208.07049.
- [18] Eslavath Ravi, Mummadi Upendra Kumar. (2022). *Android Malware Detection with Classification Based on Hybrid Analysis and N-gram Feature Extraction*. In *Advancements in Smart Computing and Information Security (ASCIS)*, Springer.
- [19] Ekta Gandotra, Divya Bansal, Sanjeev Sofat. (2014). *Malware Analysis and Classification: A Survey*. Journal of Information Security.
- [20] P. Kumar, A.A. Kumar, C. Sahayakingsly, et al. (2021). *Analysis of Intrusion Detection in Cyber Attacks Using Deep Learning Neural Networks*. Peer-to-Peer Networking and Applications, Springer.
- [21] Louis Van Langendonck, Ismael Castell-Uroz, Pere Barlet-Ros, et al. (2024). *PPT-GNN: A Practical Pre-Trained Spatio-Temporal Graph Neural Network for Network Security*. arXiv preprint.

- [22] Evan Caville, Wai Weng Lo, Siamak Layeghy, Marius Portmann. (2022). *Anomal-E: A Self-Supervised Network Intrusion Detection System based on Graph Neural Networks*. arXiv preprint.